GCD

Geospatial Data Processing

Ángel Marqués Mateu (amarques@cgf.upv.es)

Template by SLIDESGO

Table of contents

01

Vector() class

Vector entities

02

Raster() class

Image entities

03

Screen() class

Graphical User Interfaces

04

Geo-Utilities

Support functions

Vector geometries

There are 3 geometries: point, polyline and polygon

Point	Point = list of coordinates (numbers) $[x, y]$
Polyline	Polyline = list of points $[[x_0, y_0], [x_1, y_1], \ldots, [x_n, y_n]]$
Polygon	Polygon list of “parts”. Part = closed polyline $[$ $\quad [[x_{00}, y_{00}], [x_{01}, y_{01}], \ldots, [x_{0n}, y_{0n}], [x_{00}, y_{00}]],$ $\quad [[x_{10}, y_{10}], [x_{11}, y_{11}], \ldots, [x_{1n}, y_{1n}], [x_{10}, y_{10}]],$ $\quad \ldots,$ $\quad [[x_{m0}, y_{m0}], [x_{m1}, y_{m1}], \ldots, [x_{mn}, y_{mn}], [x_{m0}, y_{m0}]]$ $]$

Vector geometries

Special geometries: segment, rectangle and ray

Segment	Segment = polyline with exactly 2 vertices $[[x_0, y_0], [x_1, y_1]]$
Box or rectangle	Box = rectangle with sides parallel to coordinate axes $[x_{MIN}, y_{MIN}, x_{MAX}, y_{MAX}]$
Ray	Half line parallel to a coordinate axis ~ auxiliary entity in computing intersections $[[x_0, y_0], \ldots\ldots\ldots]$

01 ▸ Vector() class

Management (reading, writing and visualisation) of vector datasets

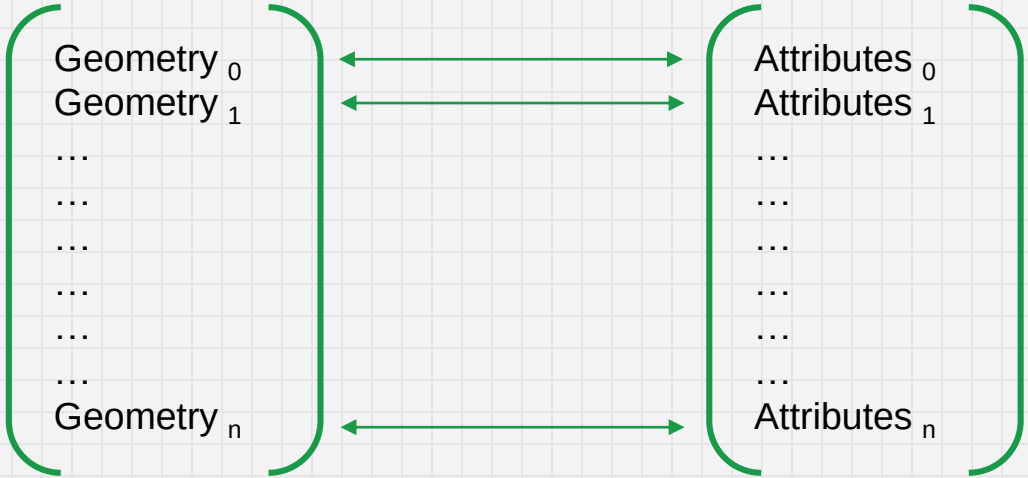
```
def __init__(self, geometry='point'):

        if geometry.upper() not in ['POINT’
           , 'POLYLINE', 'POLYGON']:
            self._geometry = None
        else:
            self._geometry = geometry

        self._source = None
        self._format = None
        self._epsg = None
        self._coordinates = []
        self._attributes = []
        self._bbox = None

        self._screen = {
            'transform': None,
            'coordinates': []
        }
```

Valid types are point, polyline or polygon

Main properties: coordinates and attributes

01 ▸ Vector() class

Relationship between geometries and attributes/properties



01 ▸ Vector() class

Management (reading, writing and visualisation) of vector datasets

`read_csv(), write_csv()`	Reads and writes comma separated values (CSV) files with arbitrary separators
`read_geojson(), write_geojson()`	Reads and writes files in GeoJSON format
`random_points()`	Creates a set of random points
`add_field(), assign()`	Adds a new field to properties table and sets the value of a given field
`project()`	Transforms coordinates between two coordinate reference systems (CRS)
`osm()`	Visualises datasets on OSM base maps
`select(), calculate()`	Selects records and calculates field values
`index()`	Creates a grid spatial index of a point cloud

01 ▸ Vector() class

Basic patterns

`import geo_2024 as geo`	Imports the GEO library
`dataset = geo.Vector()`	Creates an instance of the Vector() class
`dataset.random_points(.....)`	Updates coordinates with random points and atributes with integer IDs
`dataset.coordinates`	Gets coordinates
`dataset.attributes`	Gets attributes

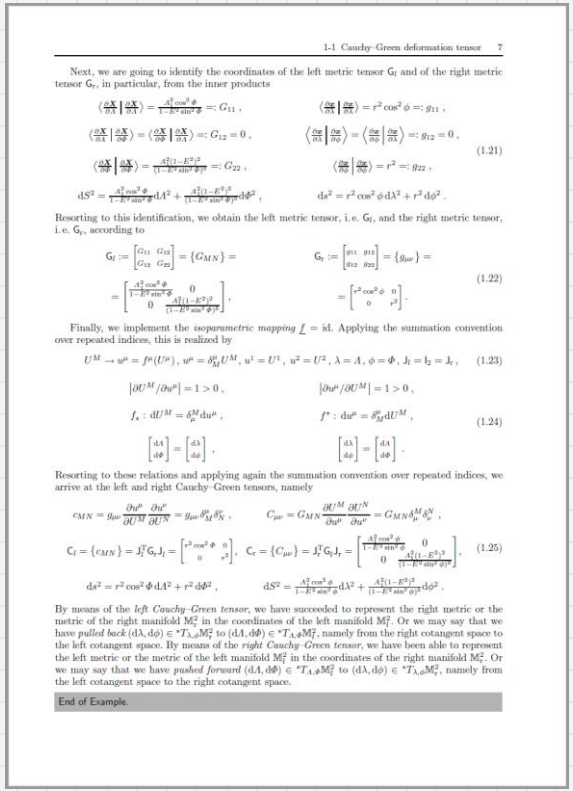
`for entity in dataset.coordinates:`
`  x, y = entity`
`  ......`
	Scans all entities in the dataset

`for count, entity in enumerate(dataset.coordinates):`
`  x, y = entity`
`  attributes = dataset[count]`
`  ......`
	Scans all entities and atributes (or porperties)

Vector geometries

Map projections

Direct problem	$x = f(\phi, \lambda, \ldots)$ $y = g(\phi, \lambda, \ldots)$
Inverse problem	$\lambda = f(x, y, \ldots)$ $\phi = g(x, y, \ldots)$
Python	import pyproj



1-1 Cauchy–Green deformation tensor 7

Next, we are going to identify the coordinates of the left metric tensor G_l and of the right metric tensor G_r, in particular, from the inner products

$$\left\langle \frac{\partial \mathbf{X}}{\partial \Lambda} \middle| \frac{\partial \mathbf{X}}{\partial \Lambda} \right\rangle = \frac{A_1^2 \cos^2\phi}{1-E^2\sin^2\phi} =: G_{11}\,, \qquad \left\langle \frac{\partial \mathbf{x}}{\partial \lambda} \middle| \frac{\partial \mathbf{x}}{\partial \lambda} \right\rangle = r^2\cos^2\phi =: g_{11}\,,$$

$$\left\langle \frac{\partial \mathbf{X}}{\partial \Lambda} \middle| \frac{\partial \mathbf{X}}{\partial \Phi} \right\rangle = \left\langle \frac{\partial \mathbf{X}}{\partial \Phi} \middle| \frac{\partial \mathbf{X}}{\partial \Lambda} \right\rangle =: G_{12} = 0\,, \qquad \left\langle \frac{\partial \mathbf{x}}{\partial \lambda} \middle| \frac{\partial \mathbf{x}}{\partial \phi} \right\rangle = \left\langle \frac{\partial \mathbf{x}}{\partial \phi} \middle| \frac{\partial \mathbf{x}}{\partial \lambda} \right\rangle =: g_{12} = 0\,, \tag{1.21}$$

$$\left\langle \frac{\partial \mathbf{X}}{\partial \Phi} \middle| \frac{\partial \mathbf{X}}{\partial \Phi} \right\rangle = \frac{A_2^2(1-E^2)^2}{(1-E^2\sin^2\phi)^2} =: G_{22}\,, \qquad \left\langle \frac{\partial \mathbf{x}}{\partial \phi} \middle| \frac{\partial \mathbf{x}}{\partial \phi} \right\rangle = r^2 =: g_{22}\,,$$

$$\mathrm{d}S^2 = \frac{A_1^2\cos^2\phi}{1-E^2\sin^2\phi}\mathrm{d}\Lambda^2 + \frac{A_2^2(1-E^2)^2}{(1-E^2\sin^2\phi)^2}\mathrm{d}\Phi^2\,, \qquad \mathrm{d}s^2 = r^2\cos^2\phi\,\mathrm{d}\lambda^2 + r^2\,\mathrm{d}\phi^2\,.$$

Resorting to this identification, we obtain the left metric tensor, i.e. G_l, and the right metric tensor, i.e. G_r, according to

$$\mathsf{G}_l := \begin{bmatrix} G_{11} & G_{12} \\ G_{12} & G_{22} \end{bmatrix} = \{G_{MN}\} = \qquad \mathsf{G}_r := \begin{bmatrix} g_{11} & g_{12} \\ g_{12} & g_{22} \end{bmatrix} = \{g_{\mu\nu}\} =$$

$$= \begin{bmatrix} \frac{A_1^2\cos^2\phi}{1-E^2\sin^2\phi} & 0 \\ 0 & \frac{A_2^2(1-E^2)^2}{(1-E^2\sin^2\phi)^2} \end{bmatrix}, \qquad = \begin{bmatrix} r^2\cos^2\phi & 0 \\ 0 & r^2 \end{bmatrix}. \tag{1.22}$$

Finally, we implement the *isoparametric mapping* $\underline{f} = \mathrm{id}$. Applying the summation convention over repeated indices, this is realized by

$$U^M \to u^\mu = f^\mu(U^\nu)\,,\ u^\mu = \delta^\mu_M U^M\,,\ \mathrm{u}^1 = U^1\,,\ \mathrm{u}^2 = U^2\,,\ \lambda = \Lambda\,,\ \phi = \Phi\,,\ J_l = \mathrm{I}_2 = J_r\,, \tag{1.23}$$

$$\left|\partial U^M/\partial u^\mu\right| = 1 > 0\,, \qquad \left|\partial u^\mu/\partial U^M\right| = 1 > 0\,,$$

$$f_*: \mathrm{d}U^M = \delta^M_\mu \mathrm{d}u^\mu\,, \qquad f^*: \mathrm{d}u^\mu = \delta^\mu_M \mathrm{d}U^M\,, \tag{1.24}$$

$$\begin{bmatrix} \mathrm{d}\Lambda \\ \mathrm{d}\Phi \end{bmatrix} = \begin{bmatrix} \mathrm{d}\lambda \\ \mathrm{d}\phi \end{bmatrix}, \qquad \begin{bmatrix} \mathrm{d}\lambda \\ \mathrm{d}\phi \end{bmatrix} = \begin{bmatrix} \mathrm{d}\Lambda \\ \mathrm{d}\Phi \end{bmatrix}.$$

Resorting to these relations and applying again the summation convention over repeated indices, we arrive at the left and right Cauchy–Green tensors, namely

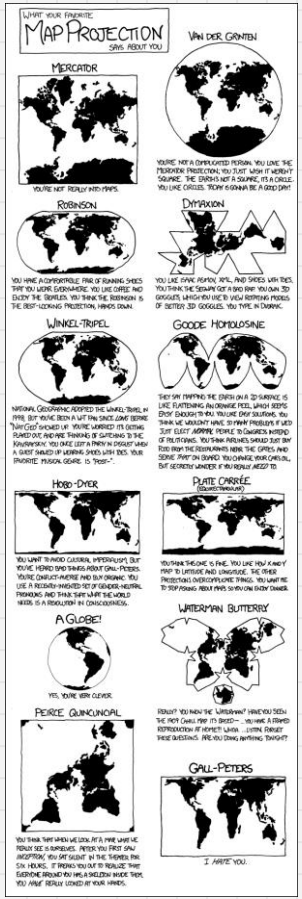
$$c_{MN} = g_{\mu\nu}\frac{\partial u^\mu}{\partial U^M}\frac{\partial u^\nu}{\partial U^N} = g_{\mu\nu}\delta^\mu_M\delta^\nu_N\,, \qquad C_{\mu\nu} = G_{MN}\frac{\partial U^M}{\partial u^\mu}\frac{\partial U^N}{\partial u^\nu} = G_{MN}\delta^M_\mu\delta^N_\nu\,,$$

$$\mathsf{C}_l = \{c_{MN}\} = J_l^T\mathsf{G}_r J_l = \begin{bmatrix} r^2\cos^2\phi & 0 \\ 0 & r^2 \end{bmatrix}, \quad \mathsf{C}_r = \{C_{\mu\nu}\} = J_r^T\mathsf{G}_l J_r = \begin{bmatrix} \frac{A_1^2\cos^2\phi}{1-E^2\sin^2\phi} & 0 \\ 0 & \frac{A_2^2(1-E^2)^2}{(1-E^2\sin^2\phi)^2} \end{bmatrix}, \tag{1.25}$$

$$\mathrm{d}s^2 = r^2\cos^2\phi\,\mathrm{d}\lambda^2 + r^2\,\mathrm{d}\phi^2\,, \qquad \mathrm{d}S^2 = \frac{A_1^2\cos^2\phi}{1-E^2\sin^2\phi}\mathrm{d}\Lambda^2 + \frac{A_2^2(1-E^2)^2}{(1-E^2\sin^2\phi)^2}\mathrm{d}\Phi^2\,.$$

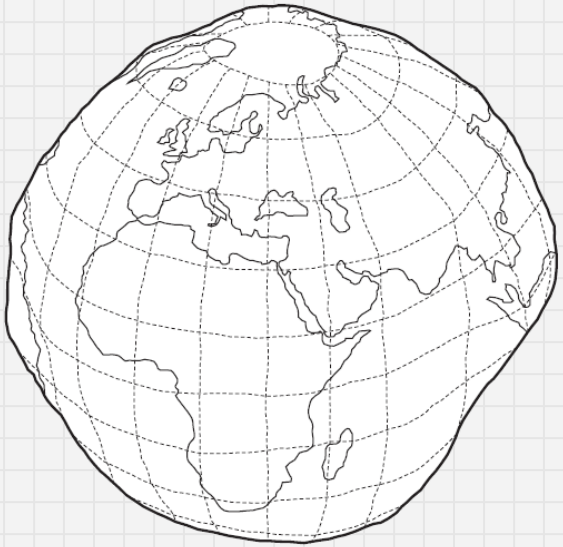
By means of the *left Cauchy–Green tensor*, we have succeeded to represent the right metric or the metric of the right manifold M_r^2 in the coordinates of the left manifold M_l^2. Or we may say that we have *pulled back* $(\mathrm{d}\Lambda, \mathrm{d}\Phi) \in {}^*T_{\lambda,\phi}\mathsf{M}_l^2$ to $(\mathrm{d}\lambda, \mathrm{d}\phi) \in {}^*T_{\lambda,\phi}\mathsf{M}_r^2$, namely from the right cotangent space to the left cotangent space. By means of the *right Cauchy–Green tensor*, we have been able to represent the left metric or the metric of the left manifold M_l^2 in the coordinates of the right manifold M_r^2. Or we may say that we have *pushed forward* $(\mathrm{d}\Lambda, \mathrm{d}\Phi) \in {}^*T_{\Lambda,\Phi}\mathsf{M}_l^2$ to $(\mathrm{d}\lambda, \mathrm{d}\phi) \in {}^*T_{\lambda,\phi}\mathsf{M}_r^2$, namely from the left cotangent space to the right cotangent space.

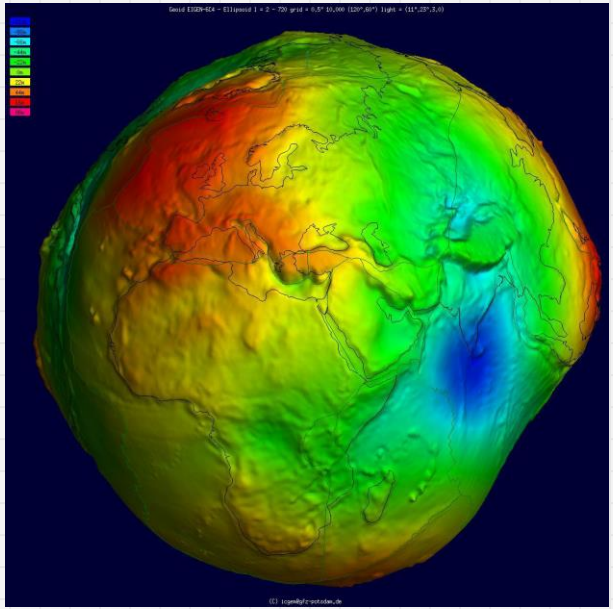
End of Example.

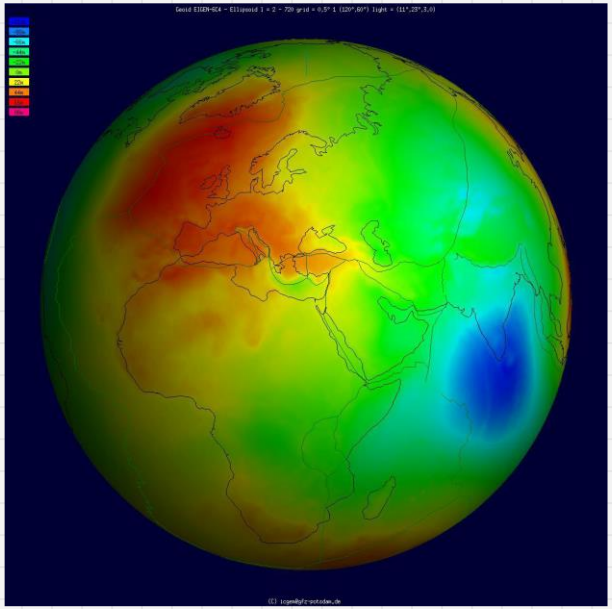


Vector geometries

Physical shape of the Earth: Geoid



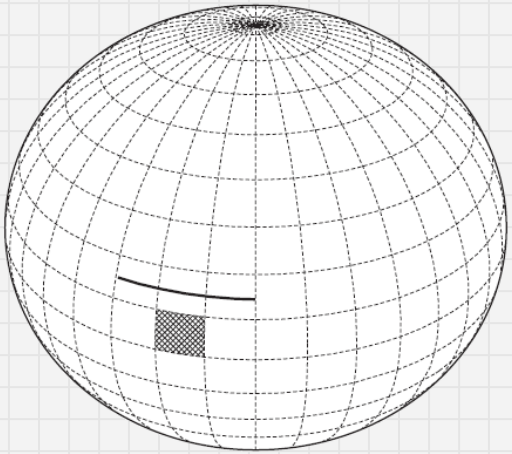


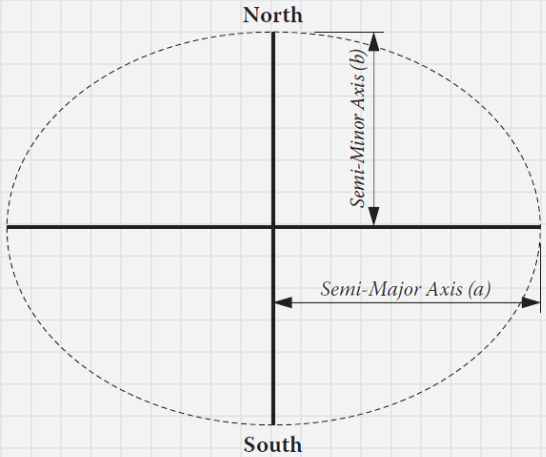


Exaggerated representation of the geoid

Vector geometries

Geometric shape of the Earth: Ellipsoid





N - S *The axis of revolution for generating the ellipsoid*

flattening $f = 1 - \frac{b}{a}$

eccentricity $e^2 = 2f - f^2$

a - *half of the major axis the semi-major axis*

b - *half of the minor axis the semi-minor axis*

Vector geometries

Reading geospatial formats

CSV	1. Comma separated values (other delimiters allowed) 2. Assume ID, X and Y fields 3. Assume there is header
GeoJSON	1. Assume well-formed contents 2. https://datatracker.ietf.org/doc/html/rfc7946 3. PSL package: *import json*
Shapefile	1. Assume well-formed contents 2. https://www.esri.com/content/dam/esrisites/sitecore-archive/Files/Pdfs/library/whitepapers/pdfs/shapefile.pdf 3. Non PSL package: *import shapefile*

GCD

Geospatial Data Processing

Ángel Marqués Mateu (amarques@cgf.upv.es)

Template by SLIDESGO